COLLEGE EVENT LISTING AND REGISTRATION

evege

Project explanation, demonstration walkthrough, and presentation notes

A practical guide to what the app does, how its parts work together, and how to explain the project clearly.

Java 8 · Spring Boot 2.7 · PostgreSQL · HTML · CSS · JavaScript

In one sentence

evege is a locally running college-events website where visitors can find events and photos, create an account, and register for events.

What this guide covers

Features, application flow, database design, account safety, how to run and demonstrate the app, likely questions, and honest current limitations.

Presentation advice: You do not need to memorize this guide. Understand the user journey, rehearse the short introduction, and describe assistance honestly if your teacher asks how the code was developed.

Prepared for the evege project · Updated October 2026

01 · THE PRODUCT

What has been built

The project turns an event-management idea into a working, branded website. The home page is designed for students who want to discover college events. The interface is responsive, uses the **evege** name, and presents events as visual cards rather than a plain list.

Discover events

Six sample events are loaded from the backend: Cricket, Football, Code Challenge, Robotics, Dance Competition, and Singing Competition.

Each event includes a category, date, venue, short description, and image.

Find the right event

Search checks event names, category names, and venues. Category controls filter the list to Sports, Technical, or Cultural events.

Explore the gallery

The gallery combines event photos with four extra community-photo entries. The Code Challenge remains listed as an upcoming event but is omitted from the gallery. Selecting a photo opens a larger detail dialog. Photos are served locally with the app.

Cricket, Football, Robotics, Dance, Singing, and other event photos were matched to their entries.

Use an account

Visitors can create an account or sign in. Registration for an event is available only after sign-in. An email cannot be registered twice for the same event.

Typical student journey

Browse View upcoming events → **Choose** Search, filter, or open a photo → **Sign in** Create an account if needed → **Register** Save a place for the event

Key design choice: The image files are bundled as local static assets in the project. The event and registration records live in PostgreSQL. That keeps photos reliable without storing large image files inside database rows.

02 · HOW IT WORKS

Application architecture

The browser displays the interface and uses HTTP requests to communicate with the Java backend. The backend applies the application rules and uses Spring Data JPA to read and write PostgreSQL records.

Browser UI
HTML, CSS, JavaScript

↔

Spring Boot API
Java controllers and validation

↔

Spring Data JPA
Java-to-database mapping

↔

PostgreSQL
Events, users, registrations

What happens when someone opens the home page?

1. Spring Boot serves the static home page and its CSS and JavaScript.
2. The browser requests `GET /api/events`.
3. `EventController` asks the event repository for events ordered by date.
4. Spring Data JPA reads the `events` table from PostgreSQL and returns JSON.
5. JavaScript renders event cards and gallery tiles using those event records and local image files.

Database records

Table / entity	What it stores	Important relationship or rule
`events` / Event	Name, category, date, venue	Listed in event-date order.
`app_users` / AppUser	Full name, unique email, password hash	Email identifies the account; the plain password is not stored.
`registrations` / Registration	Name, email, registration time, event reference	References an event; event plus email is unique.

> **Sample data:** On first start, if the events table is empty, the app creates six example events. Their dates are calculated relative to that first start date. Existing event rows are not reseeded over.

Sign-in and registration rules

The account page has sign-in and create-account forms. A successful sign-in establishes a server-managed session. The browser checks the current account to decide whether to show sign-in or account controls.

Password and request protections

- New passwords must be at least eight characters and are hashed with BCrypt before saving.
- Login checks the entered password against the saved hash; it does not retrieve a plain-text password.
- Session IDs are renewed when authentication succeeds, helping prevent session fixation.
- State-changing requests use CSRF protection. The frontend fetches a CSRF token and sends it when signing up, logging in, logging out, or registering.
- Event registration requires an authenticated account; the server enforces this rather than relying only on the screen.
- Request validation checks required names, valid email formats, and field length. Duplicate email registrations for one event are rejected.

Useful API routes

Route	Purpose
`GET /api/events`	Read upcoming events from PostgreSQL.
`GET /api/auth/csrf`	Get the token required for protected requests.
`POST /api/auth/signup`	Create an account, then start its session.
`POST /api/auth/login`	Authenticate an existing account.
`GET /api/auth/me`	Check session state; returns 204 when signed out.
`POST /api/auth/logout`	Invalidate the session and sign out.
`POST /api/events/{eventId}/registrations`	Save an event registration for a signed-in user.

> **Do not claim:** This project does not currently provide an administrator dashboard, event create/edit/delete screens, payment processing, email notifications, or cloud hosting. The included gallery images are local project files, not uploads managed through an admin panel.

04 · LIVE DEMONSTRATION

How to run and present it

Run this Windows project

In PowerShell, move into the project directory. On this machine, the portable Java/PostgreSQL launcher avoids requiring Docker, WSL, or administrator access:

```
cd "C:\Users\Ayushi Pandey\Desktop\JAVA\evege"
powershell -NoProfile -ExecutionPolicy Bypass -File .\run-evege.ps1
```

Once startup completes, open **http://localhost:8080**. Keep the launcher window open while demonstrating; press `Ctrl+C` in that window to stop it. The database files are stored locally under `D:\evege-runtime`.

A simple 2-minute demonstration

1. Show the evege home page and the event cards.
2. Search for an event or use a Sports, Technical, or Cultural category filter.
3. Open the gallery and click an image to show its event detail.
4. Select Register. Explain that sign-in is needed before registration.
5. Create an account or sign in, return to an event, and submit a registration.
6. Explain that the event list, account, and registration records are persisted in PostgreSQL.

Short presentation introduction

> "My project is evege, a college event discovery and registration website. Students can browse upcoming events, search and filter by category, and view event photos in a gallery. They can create an account and sign in before registering. The interface is built with HTML, CSS, and JavaScript. A Java Spring Boot backend exposes the APIs, and PostgreSQL stores the event, user, and registration data. I also added password hashing and CSRF protection for account-related requests."

Use wording that reflects your actual work and your course's rules. If asked, be open that you used AI assistance to help implement and explain the project, and point to the pieces you can explain yourself.

Questions your teacher may ask

Why use PostgreSQL?

It is a relational database suited to linked records. A registration points to an event, and uniqueness rules prevent duplicate signups.

What is Spring Boot doing?

It starts the Java web server, serves the static UI, exposes HTTP API routes, and connects application code to persistence and security components.

What does JPA do?

JPA maps Java entity classes such as Event and Registration to database tables. Spring Data repositories provide common database operations.

Why hash passwords?

Hashing means the database stores a one-way password hash instead of the user's original password. BCrypt is the configured password encoder.

What is CSRF protection?

It requires a valid token for state-changing browser requests, reducing the risk that another site tricks a signed-in browser into sending an unwanted request.

Where are the photos stored?

As static files inside the app's images folder. The gallery maps event names to image paths; photo data itself is not stored in PostgreSQL.

Project file map

`src/main/resources/static/index.html`, `styles.css`	Home page structure and visual styling.
`src/main/resources/static/app.js`	Loads events; implements search, filters, image mapping, gallery, and registration interactions.
`src/main/resources/static/login.html`, `login.js`	Account sign-in and account-creation interface.
`src/main/java/.../event`	Event entity, event API, repository, and first-run sample data.
`src/main/java/.../auth`	User entity, account API, login page route, and security configuration.
`src/main/java/.../registration`	Registration entity, validation, repository, and protected endpoint.
`src/main/resources/application.properties`, `pom.xml`	Database/application configuration and Java dependencies.
`run-evege.ps1`, `docker-compose.yml`	Local Windows launcher and optional Docker setup.

Future improvements to mention: An admin screen for managing events, a real image upload workflow, automated tests, and deployment beyond localhost. These are possible next steps, not features already included.

Be able to explain the path of one request: the browser calls the route, a controller handles it, a repository reads or writes an entity, and the database stores the result.